

Projet GSB Ordonnances — Playbook global

Le projet fil rouge de la 1^{re} année SLAM, du premier objet C# jusqu'à l'application CRUD complète.

Ce document retrace **tout le parcours** : POO, modélisation, base de données, puis l'application WinForms. Il ne contient **pas de code complet à recopier** : à chaque étape, tu as la signature attendue et la consigne. À toi d'écrire le corps. Les explications sont volontairement courtes — c'est un fil conducteur, pas un cours. Réfère-toi à tes notes de cours pour la théorie.

Comment lire ce playbook

- **Signature** : la déclaration exacte que tu dois produire (nom, paramètres, type de retour). À respecter au caractère près, c'est ce qui garantit la cohérence du projet.
- **Consigne** : ce que la méthode ou le composant doit faire.
-  **Flag** : un point de contrôle. Tu produis une preuve que ça marche.

Tout le projet utilise les **conventions de nommage** fixées au chapitre 0. Aucun nom ne change en cours de route.

Chapitre 0 – Conventions du projet (à respecter partout)

ÉLÉMENT	CONVENTION	EXEMPLE
Classe métier	PascalCase singulier	Patient, LignePrescription
Propriété publique	PascalCase	Nom, DateNaissance, Id
Contrôleur	PascalCase + suffixe Controller	PatientController
Méthode de contrôleur	PascalCase, verbe français	ObtenirTousLesPatients
Form	PascalCase + suffixe Form	PatientListForm
Champ privé	_camelCase	_patientController, _lignes
Méthode privée	PascalCase	ChargerPatients
Variable / paramètre	camelCase	motCle
Contrôle WinForms	préfixe de type + camelCase	txtNom, btnEnregistrer, dgvPatients, cmbMedecin
Handler d'événement	nomControle_Evenement (généré par Visual Studio, minuscule)	btnRechercher_Click
Table SQL	MAJUSCULES	PATIENT
Colonne SQL	camelCase	numPatient

Trois règles d'or transversales :

1. **Un nom ne change jamais.** Une méthode créée garde son nom toute l'année. Pour étendre, on ajoute une méthode ou un paramètre, on ne renomme pas.
2. **Toute donnée utilisateur entrant dans une requête passe par un paramètre.** Jamais de concaténation.
3. **La propriété C# Id correspond à la colonne SQL numXxx** (Patient.Id ↔ numPatient). Le mapping se fait dans le contrôleur.

Namespace racine du projet : GSB.Ordonnances

Chaîne de connexion : Server=localhost;Port=3306;Database=gsb_ordonnances;Uid=gsb;Pwd=gsbpass;

Chapitre 1 – Modèle objet (POO en C#)

On commence par les classes métier, sans base de données. Ces classes sont le **socle** : contrôleurs et formulaires manipuleront ces objets. Quatre concepts de POO sont mobilisés, à reconnaître :

- **Encapsulation** : les données sont des propriétés (get / set), jamais des champs publics bruts. On contrôle l'accès.
- **Héritage** : Patient et Medecin héritent d'une classe commune Personne qui factorise nom, prénom, date de naissance.

- **Polymorphisme** : la méthode `Presentation()` est redéfinie (`override`) différemment selon la classe — un patient se présente autrement qu'un médecin.
- **Composition** : une `Ordonnance` est composée de `LignePrescription` , elles-mêmes composées d'un `Medicament` . Détruire l'ordonnance détruit ses lignes.

1.1 — La classe abstraite `Personne`

Consigne : crée une classe **abstraite** `Personne` qui factorise ce qui est commun à un patient et un médecin.

MEMBRE	TYPE	RÔLE
<code>Id</code>	<code>int</code> (propriété)	Identifiant, mappé plus tard sur la PK SQL
<code>Nom</code>	<code>string</code> (propriété)	—
<code>Prenom</code>	<code>string</code> (propriété)	—
<code>DateNaissance</code>	<code>DateTime</code> (propriété)	—

MÉTHODE	SIGNATURE	CONSIGNE
Constructeur	<code>Personne(string nom, string prenom, DateTime dateNaissance)</code>	Initialise les trois propriétés
Âge	<code>int CalculerAge()</code>	Retourne l'âge en années à partir de <code>DateNaissance</code> . Attention : ce n'est pas <code>DateTime.Now.Year - DateNaissance.Year</code> , il faut tenir compte du mois et du jour (sinon quelqu'un né en décembre est compté trop vieux en début d'année).
Présentation	<code>virtual string Presentation()</code>	Retourne <code>"Prenom NOM"</code> . virtual pour être redéfinie par les classes filles

À quoi ressemble un constructeur ? Un constructeur porte le nom de la classe, n'a pas de type de retour, et sert à remplir les propriétés à la création de l'objet. Schéma illustratif (à compléter toi-même) :

```
public class Patient
{
    public string Nom { get; set; }

    public Patient(string nom, ...) // ← même nom que la classe
    {
        Nom = nom;                // on remplit la propriété
        ...
    }
}
```

Les `...` sont à remplacer par les autres champs. Ce n'est pas le code GSB final, juste la forme.

1.2 — Patient (hérite de Personne)

Consigne : `Patient` ajoute le numéro de sécu et redéfinit la présentation.

MEMBRE	TYPE	RÔLE
<code>NumeroSecu</code>	<code>string</code> (propriété)	13 caractères dans ce projet

MÉTHODE	SIGNATURE	CONSIGNE
Constructeur	<code>Patient(string nom, string prenom, DateTime dateNaissance, string numeroSecu)</code>	Appelle <code>: base(...)</code> puis initialise <code>NumeroSecu</code>
Majorité	<code>bool EstMajeur()</code>	<code>true</code> à partir de 18 ans (réutilise <code>CalculerAge</code>)
Présentation	<code>override string Presentation()</code>	Retourne <code>"Prenom NOM"</code> adapté au patient

1.3 — Medecin (hérite de Personne)

MEMBRE	TYPE	RÔLE
<code>NumeroRPPS</code>	<code>string</code>	Identifiant ordinal du médecin, 11 caractères
<code>Specialite</code>	<code>string</code>	—
<code>MotDePasse</code>	<code>string</code>	Stockera un hash , jamais un mot de passe en clair

MÉTHODE	SIGNATURE	CONSIGNE
Constructeur	<code>Medecin(string nom, string prenom, DateTime dateNaissance, string numeroRPPS, string specialite, string motDePasse)</code>	<code>: base(...)</code> + initialise le reste
Présentation	<code>override string Presentation()</code>	Retourne <code>"Dr NOM, Specialite"</code>

1.4 — Medicament et Allergie

Deux classes simples, sans héritage.

CLASSE	PROPRIÉTÉS	CONSTRUCTEUR
<code>Medicament</code>	<code>Id (int), Nom (string), Dosage (string)</code>	<code>Medicament(string nom, string dosage)</code>
<code>Allergie</code>	<code>Id (int), Libelle (string)</code>	<code>Allergie(string libelle)</code>

1.5 — LignePrescription (composition)

Consigne : une ligne de prescription **contient** un médicament, plus une posologie et une durée. C'est de la **composition** : une ligne n'existe pas sans son médicament.

MEMBRE	TYPE
Medicament	Medicament (propriété)
Posologie	string
DureeJours	int

Constructeur: `LignePrescription(Medicament medicament, string posologie, int dureeJours)`

1.6 — Ordonnance (agrégation + composition)

Consigne : une ordonnance relie un médecin, un patient, et possède une **liste de lignes de prescription**.

MEMBRE	TYPE	RÔLE
Id	int	mappé sur numOrdonnance
DateEmission	DateTime	—
Prescripteur	Medecin	le médecin
Patient	Patient	le destinataire
Lignes	List<LignePrescription>	en lecture seule de l'extérieur

MÉTHODE	SIGNATURE	CONSIGNE
Constructeur	<code>Ordonnance(Medecin prescripteur, Patient patient)</code>	Initialise la liste vide, <code>DateEmission = DateTime.Now</code>
Ajout de ligne	<code>void AjouterLigne(LignePrescription ligne)</code>	Ajoute à la liste interne

Le point qui fait tilt : `LignePrescription` (modèle objet) correspondra à la table `CONTENIR` (base de données). Le monde POO et le monde Merise nomment la même réalité différemment. Garde ça en tête.

Plus largement, ne confonds jamais **classe** (modèle objet, chapitre 1) et **entité** (modèle conceptuel, chapitre 2). Une classe a des méthodes et du comportement ; une entité n'a que des attributs. La classe `Patient` fait des choses (`EstMajeur()`), l'entité `PATIENT` stocke des données. On passe constamment de l'un à l'autre dans ce projet : sache toujours dans quel monde tu te trouves.

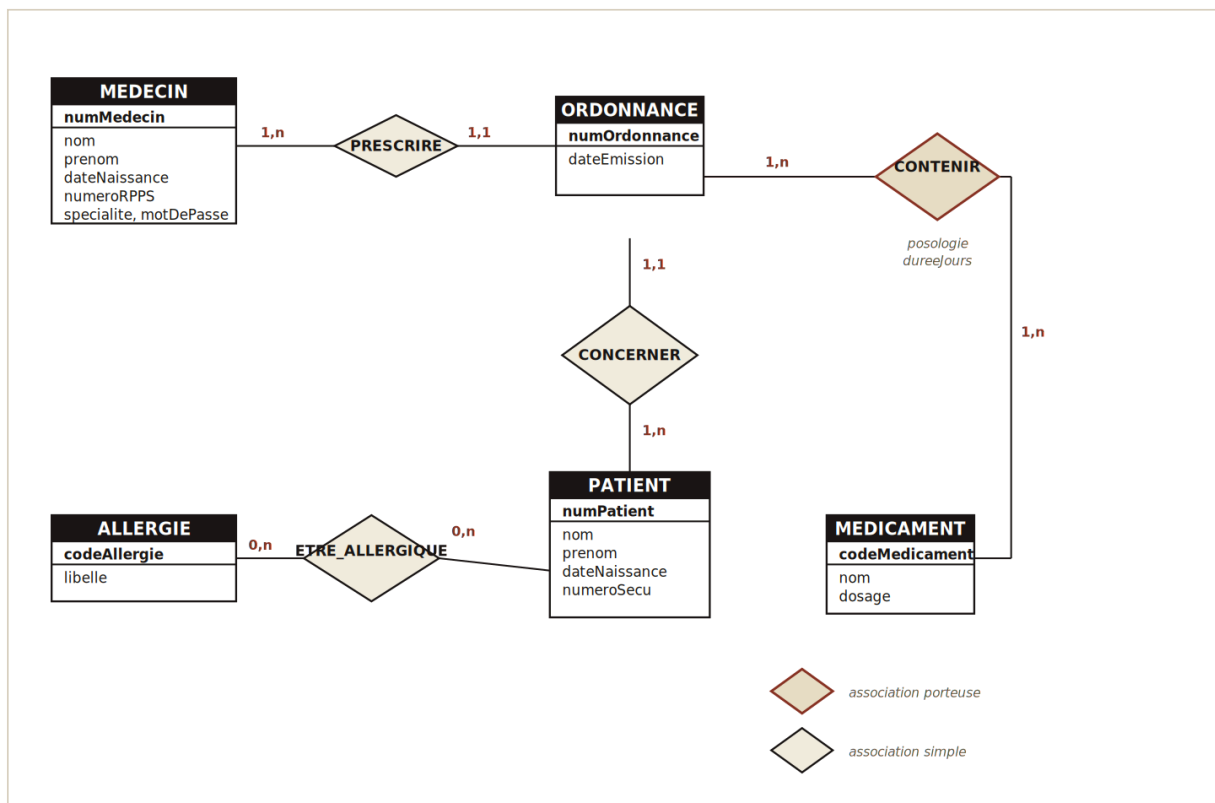
🚩 Flag A1 — Checkpoint POO

Dans un `Main` de test, crée un patient de 25 ans et un de 16 ans. Affiche `Console.WriteLine($"FLAG-A1- {(patient25.EstMajeur() ? 1 : 0)}{(patient16.EstMajeur() ? 1 : 0)}")`. Rends la valeur.

Chapitre 2 – Modèle conceptuel (MCD)

Avant la base, le schéma conceptuel Merise. Pas de code ici, du modèle.

Le MCD du projet



MCD du projet GSB Ordonnances

Les attributs en italique sous CONTENIR (*posologie* , *dureeJours*) sont **portés par l'association** : ils dépendent du couple (ordonnance, médicament), pas de l'un ou de l'autre seul. C'est ce qui fait de CONTENIR une **association porteuse** (losange foncé). Une ordonnance contient plusieurs médicaments, chacun avec sa propre posologie et sa propre durée.

2.1 – Les entités

Cinq entités, avec leurs attributs et identifiants :

ENTITÉ	IDENTIFIANT	ATTRIBUTS
PATIENT	numPatient	nom, prenom, dateNaissance, numeroSecu
MEDECIN	numMedecin	nom, prenom, dateNaissance, numeroRPPS, specialite, motDePasse
ORDONNANCE	numOrdonnance	dateEmission
MEDICAMENT	codeMedicament	nom, dosage
ALLERGIE	codeAllergie	libelle

2.2 — Les associations

ASSOCIATION	ENTITÉS RELIÉES	CARDINALITÉS	PORTEUSE ?
PRESCRIRE	MEDECIN — ORDONNANCE	(1,n) — (1,1)	non
CONCERNER	ORDONNANCE — PATIENT	(1,1) — (1,n)	non
CONTENIR	ORDONNANCE — MEDICAMENT	(1,n) — (1,n)	oui : posologie, dureeJours
ETRE_ALLERGIQUE	PATIENT — ALLERGIE	(0,n) — (0,n)	non

2.3 — Points de vigilance Merise

- **Pas d'héritage en MCD** : on ne factorise pas PATIENT et MEDECIN dans une entité PERSONNE. La duplication d'attributs (nom, prenom, dateNaissance) est **assumée** côté conceptuel, contrairement à la POO où on a une classe `Personne`.
- **CONTENIR est porteuse** : elle porte `posologie` et `dureeJours`, car ces données dépendent du couple (ordonnance, médicament), pas de l'un ou de l'autre seul.
- **Cardinalités à savoir lire** : `(1,n)` côté MEDECIN sur PRESCRIRE signifie « un médecin prescrit *de une à plusieurs* ordonnances ». `(1,1)` côté ORDONNANCE signifie « une ordonnance est prescrite par *exactement un* médecin ». Le premier chiffre est le minimum, le second le maximum.
- Le `numeroSecu` est **simplifié à 13 caractères** dans ce projet (15 dans la réalité). C'est une simplification pédagogique assumée.

Flag B1 — MCD

Sur ton MCD, identifie l'association porteuse et justifie en une phrase pourquoi `posologie` ne peut être un attribut ni de ORDONNANCE ni de MEDICAMENT. Rends ta justification.

Chapitre 3 — Du MCD au MLD, MySQL et Docker

3.1 — Règles de passage MCD → MLD (rappel)

- Une **entité** devient une **table**, son identifiant devient la **clé primaire**.
- Une association **(1,1) — (1,n)** : la clé de l'entité côté (1,n) migre comme **clé étrangère** côté (1,1). Pas de table d'association. (Exemple : *PRESCRIRE* et *CONCERNER* deviennent les colonnes `numMedecin` et `numPatient` dans *ORDONNANCE*.)
- Une association **(1,n) — (1,n)** : devient une **table de jointure** avec une clé primaire composite des deux clés étrangères. Les attributs porteurs deviennent des colonnes de cette table. (Exemple : *CONTENIR* devient une table avec PK `(numOrdonnance, codeMedicament)` + les colonnes `posologie` et `dureeJours`.)

Pourquoi pas d'héritage en MLD ? En POO, `Patient` et `Medecin` héritent de `Personne`. En relationnel, il n'y a pas d'héritage : on a deux tables `PATIENT` et `MEDECIN` qui **dupliquent** les colonnes `nom`, `prenom`, `dateNaissance`. Cette duplication est assumée. C'est l'une des différences fondamentales entre le monde objet et le monde relationnel.

3.2 — Le schéma relationnel cible

TABLE	COLONNES CLÉS	REMARQUES
PATIENT	numPatient (PK auto)	numeroSecu UNIQUE
MEDECIN	numMedecin (PK auto)	numeroRPPS UNIQUE
ORDONNANCE	numOrdonnance (PK auto)	numMedecin FK, numPatient FK, ON DELETE RESTRICT
MEDICAMENT	codeMedicament (PK auto)	—
ALLERGIE	codeAllergie (PK auto)	libelle UNIQUE
CONTENIR	(numOrdonnance, codeMedicament) PK composite	porte posologie, dureeJours ; FK ord ON DELETE CASCADE , FK med ON DELETE RESTRICT
ETRE_ALLERGIQUE	(numPatient, codeAllergie) PK composite	FK patient ON DELETE CASCADE , FK allergie ON DELETE RESTRICT

Logique des ON DELETE : supprimer une ordonnance supprime ses lignes (CASCADE). Supprimer un patient qui a des ordonnances est **interdit** (RESTRICT) — on protège les données médicales.

3.3 — Script SQL d'initialisation

Consigne : écris `01-init.sql` qui crée les 7 tables dans le bon ordre (parents avant enfants), avec PK, FK, contraintes UNIQUE et NOT NULL, et les `ON DELETE` ci-dessus. Ajoute un jeu de test minimal :

- Patients : Marie DUPONT, Jean LEROY
- Médecins : Dr DURAND (généraliste), Dr MARTIN
- Quelques médicaments (ex. Doliprane 500 mg, Amoxicilline 1 g) et allergies (ex. Pénicilline, Aspirine)

- Au moins une ordonnance pour Marie DUPONT avec deux lignes (sinon le test de suppression FK n'aura rien à refuser, et le maître-détail du chapitre 8 n'aura rien à afficher)

Ordre de création obligatoire : une table ne peut pas référencer une table qui n'existe pas encore. Crée donc d'abord les tables « parents » (PATIENT, MEDECIN, MEDICAMENT, ALLERGIE), puis ORDONNANCE (qui référence MEDECIN et PATIENT), enfin les tables de jointure CONTENIR et ETRE_ALLERGIQUE. Même logique pour les INSERT du jeu de test : un patient avant son ordonnance, une ordonnance avant ses lignes.

3.4 — Docker Compose

Consigne : écris `docker-compose.yml` avec deux services :

SERVICE	IMAGE	PORT	NOTES
mysql	mysql:8.0	3306	volume nommé <code>mysql_data</code> , script monté dans <code>/docker-entrypoint-initdb.d/</code>
phpmyadmin	phpmyadmin:latest	8080	pointe sur le service mysql

Variables d'environnement : `MYSQL_ROOT_PASSWORD=rootpassword`, `MYSQL_DATABASE=gsb_ordonnances`, `MYSQL_USER=gsb`, `MYSQL_PASSWORD=gsbpass`.

Démarrage : `docker compose up -d`. Vérifie dans phpMyAdmin (`localhost:8080`) que les tables et le jeu de test sont présents.

Pièges Docker fréquents :

- « *Unable to connect to any of the specified MySQL hosts* » → le conteneur n'est pas démarré. Lance `docker compose up -d`.
- « *Access denied for user 'gsb'* » → les identifiants de la chaîne de connexion ne correspondent pas à ceux du `docker-compose.yml`. Vérifie `gsb` / `gsbpass`.
- « *Unknown database 'gsb_ordonnances'* » → le script SQL n'a pas été rejoué après une modif. Relance proprement avec `docker compose down -v` (le `-v` efface le volume) puis `docker compose up -d`.

Le script `01-init.sql` n'est exécuté **qu'à la première création** du volume. Si tu le modifies, il faut détruire le volume pour qu'il soit rejoué.

Flag C1 — Base opérationnelle

Une fois la base montée, exécute dans phpMyAdmin une requête qui compte les ordonnances de Marie DUPONT par un JOIN ORDONNANCE/PATIENT. Rends le nombre obtenu.

Chapitre 4 — Premier contrôleur MVC

On bascule dans l'application WinForms. Architecture MVC à respecter pour tout le reste du projet.

4.1 — Arborescence du projet

```
GSB.Ordonnances/  
├─ Controllers/      logique métier  
├─ DataAccess/       accès base  
├─ Models/           classes du chapitre 1  
├─ Views/            formulaires WinForms  
└─ Program.cs
```

4.2 — La couche d'accès aux données

Consigne : crée `DataAccess/DbConnexion.cs`, classe statique.

MÉTHODE	SIGNATURE	CONSIGNE
Ouvrir	<code>static MySqlConnection Ouvrir()</code>	Crée une <code>MySqlConnection</code> avec la chaîne du chapitre 0, l'ouvre, la retourne. L'appelant gère la fermeture via <code>using</code> .

Installe le paquet NuGet `MySQL.Data` au préalable.

4.3 — Le premier contrôleur

Consigne : crée `Controllers/PatientController.cs`.

MÉTHODE	SIGNATURE	CONSIGNE
Lecture	<code>List<Patient></code> <code>ObtenirTousLesPatients()</code>	SELECT sur PATIENT trié par nom, prénom. Mappe chaque ligne en <code>Patient</code> (n'oublie pas <code>p.Id = ...GetInt32("numPatient")</code>). Utilise le triple <code>using</code> (connexion, commande, reader).

À quoi ressemble une méthode de contrôleur ? Trois temps : une requête SQL en texte, l'exécution, et le mapping ligne → objet. Schéma illustratif (réduit à un seul champ lu) :

```
public List<Patient> ObtenirTousLesPatients()
{
    var liste = new List<Patient>();
    string sql = "SELECT numPatient, nom, ... FROM PATIENT";
    using (var connexion = DbConnexion.Ouvrir())
    using (var commande = new MySqlCommand(sql, connexion))
    using (var reader = commande.ExecuteReader())
    {
        while (reader.Read())
        {
            var p = new Patient(reader.GetString("nom"), ...);
            p.Id = reader.GetInt32("numPatient"); // mapping colonne → propriété
            liste.Add(p);
        }
    }
    return liste;
}
```

Les `...` sont à compléter (les autres colonnes, les autres champs du constructeur). La structure, elle, ne change pas d'une lecture à l'autre.

4.4 — Pourquoi le triple `using` ?

Une connexion à la base est une **ressource** qui doit être fermée après usage, même si une erreur survient. Le bloc `using` garantit cette fermeture automatique à la sortie. On en imbrique trois (connexion, commande, reader) car les trois sont des ressources à libérer.

Ce pattern revient à **chaque** lecture (tu viens d'en voir la forme complète au 4.3). Seul le contenu du `while` — le mapping colonne → objet — change selon la table ; la structure des trois `using` reste identique partout.

4.5 — La vue : `PatientListForm`

Consigne : crée `Views/PatientListForm.cs` avec :

CONTRÔLE	TYPE	RÔLE
<code>dgvPatients</code>	<code>DataGridView</code>	affiche la liste

MEMBRE	SIGNATURE	CONSIGNE
Champ	<code>_patientController</code> (type <code>PatientController</code>)	Note : on le préfixe <code>_patient...</code> dès maintenant, car d'autres contrôleurs vont arriver.
Méthode privée	<code>void ChargerPatients()</code>	appelle <code>ObtenirTousLesPatients</code> et l'affecte à <code>dgvPatients.DataSource</code>
Événement Load	câblé sur <code>ChargerPatients</code>	charge la grille à l'ouverture

Convention posée ici : le champ contrôleur s'appelle `_patientController`, **pas** `_controller`. On anticipe l'arrivée des autres contrôleurs pour ne jamais avoir à renommer.

Piège DataGridView : si la grille affiche des colonnes en double ou des colonnes techniques indésirables, désactive `AutoGenerateColumns` ou règle les colonnes à la main. Et si une ligne vide apparaît en bas (la ligne d'ajout), mets `dgvPatients.AllowUserToAddRows = false`. Pense aussi à `ReadOnly = true` sur la grille : on édite via les formulaires, pas directement dans les cellules.

🚩 Flag A2 — Checkpoint MVC

Après ton appel à `ObtenirTousLesPatients()`, ajoute `Console.WriteLine($"FLAG-A2-{patients.Count * 13 + 7}")`. Rends la valeur.

Chapitre 5 — Recherche paramétrée et injection SQL

On rend la liste filtrable, et on en profite pour traiter la sécurité (OWASP A03).

5.1 — Étape volontairement vulnérable (à des fins pédagogiques)

Consigne : crée une méthode `RechercherParNom_Vulnérable(string motCle)` qui construit la requête **par concaténation** de la saisie utilisateur. L'underscore dans le nom est **volontaire** : il signale visuellement une méthode à ne jamais réutiliser en production.

MÉTHODE	SIGNATURE
Recherche vulnérable	<code>List<Patient> RechercherParNom_Vulnérable(string motCle)</code>

La requête vulnérable : la saisie utilisateur est collée directement dans le texte SQL.

```
string sql = "SELECT * FROM PATIENT WHERE nom LIKE '%" + motCle + "%'";
```

Si `motCle` vaut `Dupont`, la requête finale est saine : `... WHERE nom LIKE '%Dupont%'`. Mais l'utilisateur contrôle `motCle`, et il peut y mettre **autre chose que du texte**.

Teste-la, puis **attaque-la** : saisis `' OR '1'='1'` dans le champ de recherche.

Ce que MySQL reçoit alors :

```
SELECT * FROM PATIENT WHERE nom LIKE '%' OR '1'='1'
```

La condition `'1'='1'` est toujours vraie, l'opérateur `OR` court-circuite le filtre, et **toute la table** revient. La saisie de l'utilisateur a été interprétée comme du **code SQL**, pas comme une donnée. C'est une injection SQL.

5.2 — La correction : requête paramétrée

Consigne : crée la version saine `RechercherParNom(string motCle)` qui utilise un paramètre `@motCle` au lieu de la concaténation. Branche le bouton de recherche sur **cette** version. Garde l'autre dans le code comme contre-exemple, mais **plus aucun bouton ne l'appelle**.

MÉTHODE	SIGNATURE	CONSIGNE
Recherche saine	<code>List<Patient></code> <code>RechercherParNom(string motCle)</code>	<code>WHERE nom LIKE @motCle</code> , paramètre passé via <code>AddWithValue</code>

La requête paramétrée : le texte SQL ne contient **aucune** donnée utilisateur, seulement un emplacement nommé `@motCle`. La valeur est transmise séparément.

```
string sql = "SELECT * FROM PATIENT WHERE nom LIKE @motCle";  
commande.Parameters.AddWithValue("@motCle", "%" + motCle + "%");
```

Le moteur traite `@motCle` comme une **valeur littérale**, jamais comme du code. Si l'utilisateur saisit `' OR '1'='1`, MySQL cherche littéralement un patient dont le nom contient cette chaîne — et n'en trouve aucun. L'injection est neutralisée. **La règle : le plan de la requête et les données voyagent séparément.**

5.3 — Recherche combinée nom + allergie

Consigne : ajoute un `AllergieController` et une recherche croisée. **On garde le nom complet** `RechercherParNomEtAllergie`, on ne le raccourcit jamais.

CLASSE / MÉTHODE	SIGNATURE	CONSIGNE
<code>AllergieController.ObtenirLibelles</code>	<code>List<string> ObtenirLibelles()</code>	SELECT des libellés d'allergie pour remplir la combo
<code>PatientController.RechercherParNomEtAllergie</code>	<code>List<Patient></code> <code>RechercherParNomEtAllergie(string motCle, string libelleAllergie)</code>	JOIN PATIENT / ETRE_ALLERGIQUE / ALLERGIE, deux paramètres

Contrôles à ajouter à `PatientListForm` : `txtRecherche`, `btnRechercher`, `btnReset`, `cmbAllergie`, plus les labels. Champ privé `_allergieController`.

La requête à jointures : pour croiser patients et allergies, il faut relier trois tables via la table de jointure ETRE_ALLERGIQUE .

```
SELECT DISTINCT p.numPatient, p.nom, p.prenom, p.dateNaissance, p.numeroSecu
FROM PATIENT p
JOIN ETRE_ALLERGIQUE ea ON ea.numPatient = p.numPatient
JOIN ALLERGIE a ON a.codeAllergie = ea.codeAllergie
WHERE p.nom LIKE @motCle AND a.libelle = @libelle
```

Deux paramètres (@motCle , @libelle), deux JOIN pour remonter de PATIENT jusqu'à ALLERGIE en passant par la table d'association. Le DISTINCT évite les doublons si un patient a plusieurs allergies correspondantes.

5.4 — Le référentiel OWASP

À retenir pour l'épreuve E7 :

- **OWASP** : fondation qui publie le **Top 10** des risques applicatifs, mis à jour tous les 4 ans, basé sur des centaines de milliers d'applications réelles.
- **A03 — Injection** : une donnée utilisateur interprétée comme du code par un moteur (SQL, mais aussi NoSQL, LDAP, XSS...). 94 % des applications testées présentent une forme d'injection.
- **Parade** : requête paramétrée d'abord, validation des entrées en complément.

À garder en tête pour l'oral E6 comme pour l'écrit E7 : la validation des entrées (vérifier qu'un champ ne contient que des chiffres, par exemple) est **utile mais insuffisante** seule — elle peut être contournée. Le paramétrage, lui, est structurel : le moteur ne peut pas confondre donnée et code. On fait donc **les deux**, mais le paramétrage est non négociable. Note aussi que l'injection ne touche pas que SQL : le même principe vaut pour NoSQL (projet de 2^e année), les commandes système, ou le XSS côté web.

Flag D1 — Investigation OWASP

Sur le site officiel OWASP, page A03 (édition 2021), trouve le CWE le plus fréquent mappé à A03. Rends son numéro (CWE-XX) et son nom.

Chapitre 6 — CRUD Patient

L'application devient utilisable : créer, modifier, supprimer des patients. Le contrôleur passe de la simple lecture (SELECT) à l'écriture (INSERT , UPDATE , DELETE). CRUD = Create, Read, Update, Delete.

Côté interface, on introduit le **formulaire modal** : une fenêtre de saisie qui s'ouvre par-dessus la liste, la bloque, et renvoie un DialogResult (OK ou Cancel) à sa fermeture. La liste décide ensuite de rafraîchir sa grille ou non selon ce résultat. C'est le motif standard pour toute saisie de données dans une application WinForms.

6.1 — Le formulaire d'édition

Consigne : crée Views/PatientEditForm.cs , fenêtre modale d'édition d'un patient.

Contrôles : txtNom , txtPrenom , txtNumeroSecu , dtpDateNaissance , leurs labels, et deux boutons btnEnregistrer (DialogResult = OK) et btnAnnuler (DialogResult = Cancel).

Propriétés form : `FormBorderStyle = FixedDialog`, `StartPosition = CenterParent`, `AcceptButton = btnEnregistrer`, `CancelButton = btnAnnuler`.

CONSTRUCTEUR	SIGNATURE	MODE
Création	<code>PatientEditForm()</code>	champs vides, titre « Nouveau »
Modification	<code>PatientEditForm(Patient patient)</code> <code>: this()</code>	champs pré-remplis, titre « Modification »

Le `: this()` appelle le constructeur sans paramètre d'abord, pour ne pas dupliquer l'initialisation. Même principe que `: base(...)` entre parent et enfant, mais ici entre deux constructeurs de la même classe.

À quoi ressemble une surcharge de constructeur ? Deux constructeurs de même nom, distingués par leurs paramètres. Schéma illustratif :

```
public PatientEditForm()           // mode création
{
    InitializeComponent();
    ...
}

public PatientEditForm(Patient p) : this() // mode modif
{
    // : this() a déjà fait l'init ci-dessus
    txtNom.Text = p.Nom;           // on pré-remplit
    ...
}
```

C'est le compilateur qui choisit le bon constructeur selon que tu écris `new PatientEditForm()` ou `new PatientEditForm(monPatient)`.

HANDLER	SIGNATURE	CONSIGNE
Enregistrer	<code>btnEnregistrer_Click(object, EventArgs)</code>	Valide (nom non vide, numéro de sécu = 13 caractères). Si invalide : message + <code>DialogResult = None</code> pour annuler la fermeture. Sinon, selon le mode, appelle <code>AjouterPatient</code> ou <code>ModifierPatient</code> .

La validation se fait avant l'écriture en base. On vérifie d'abord la saisie côté formulaire (champ vide, longueur du numéro de sécu, format de la date). Ce n'est qu'une fois ces contrôles passés qu'on appelle le contrôleur. La validation côté interface améliore l'expérience (message immédiat, clair), mais ne remplace pas les contraintes de la base (`NOT NULL`, `UNIQUE`) qui sont l'ultime garde-fou – d'où le `try/catch` autour de l'appel au contrôleur, qui rattrape une éventuelle `MySqlException` (par exemple un numéro de sécu déjà existant, code 1062).

Concept clé DialogResult + ShowDialog : la liste ouvre cette form avec `ShowDialog(this)` (modale, bloquante). Si le retour est `DialogResult.OK`, la liste rafraîchit sa grille ; si `Cancel`, elle ne fait rien. Mettre `DialogResult = None` dans un handler **empêche** la fermeture (utile quand la validation échoue).

6.2 – Création

MÉTHODE CONTRÔLEUR	SIGNATURE	CONSIGNE
<code>PatientController.AjouterPatient</code>	<code>int</code> <code>AjouterPatient(Patient p)</code>	INSERT paramétré, puis récupère l'id via <code>LAST_INSERT_ID()</code> (avec <code>ExecuteScalar()</code>). Retourne l'id.

L'INSERT paramétré : on liste les colonnes, on met un paramètre par valeur. La colonne `numPatient` n'est **pas** dans la liste – MySQL la génère (`AUTO_INCREMENT`).

```
INSERT INTO PATIENT (nom, prenom, dateNaissance, numeroSecu)
VALUES (@nom, @prenom, @dateNaissance, @numeroSecu);
SELECT LAST_INSERT_ID();
```

La seconde requête `LAST_INSERT_ID()` renvoie l'identifiant que MySQL vient de générer. On le récupère côté C# avec `ExecuteScalar()` (qui retourne une seule valeur) et on le convertit en `int`.

Côté PatientListForm : ajoute `btnNouveau`. Son handler ouvre `PatientEditForm()` en modale ; si retour OK, rafraîchit la grille.

Trois façons d'exécuter une requête – à savoir distinguer :

MÉTHODE	POUR	RETOURNE
<code>ExecuteReader()</code>	un SELECT multi-lignes	un lecteur à parcourir
<code>ExecuteScalar()</code>	une valeur unique (id, COUNT...)	un object (la 1 ^{re} cellule)
<code>ExecuteNonQuery()</code>	INSERT / UPDATE / DELETE sans retour	le nombre de lignes affectées

6.3 — Modification

MÉTHODE CONTRÔLEUR	SIGNATURE	CONSIGNE
<code>PatientController.ModifierPatient</code>	<code>void</code> <code>ModifierPatient(Patient p)</code>	UPDATE paramétré avec <code>WHERE numPatient = @id</code> (sans le WHERE, tu écrases toute la table). Vérifie que <code>ExecuteNonQuery</code> retourne 1.

L'UPDATE paramétré : la clause `WHERE` est **vitale**.

```
UPDATE PATIENT
SET nom = @nom, prenom = @prenom,
    dateNaissance = @dateNaissance, numeroSecu = @numeroSecu
WHERE numPatient = @id
```

Sans le `WHERE numPatient = @id`, l'UPDATE s'applique à toutes les lignes : tous les patients de la base prendraient le même nom. `ExecuteNonQuery()` retournerait alors le nombre total de patients (par exemple 10) au lieu de 1. C'est l'erreur la plus dangereuse de cette séance.

Côté `PatientListForm` : câble l'événement `dgvPatients_CellDoubleClick`. Il récupère le patient via `DataBoundItem`, ouvre `PatientEditForm(patient)`, rafraîchit si OK.

`DataBoundItem` donne l'objet métier complet (y compris l'Id, même s'il n'est pas affiché) sous la ligne double-cliquée. Bien plus fiable que de relire les cellules une à une.

6.4 — Suppression

MÉTHODE CONTRÔLEUR	SIGNATURE	CONSIGNE
<code>PatientController.SupprimerPatient</code>	<code>void</code> <code>SupprimerPatient(int id)</code>	DELETE paramétré par id.

Côté `PatientListForm` : ajoute `btnSupprimer`. Son handler demande confirmation (`MessageBox Yes/No`), puis supprime. **Intercepte la `MySqlException` de code 1451** (violation FK) pour afficher un message clair : « ce patient a des ordonnances, supprime-les d'abord ». Le `ON DELETE RESTRICT` du chapitre 3 produit cette erreur.

Le DELETE paramétré :

```
DELETE FROM PATIENT WHERE numPatient = @id
```

Si ce patient a des ordonnances, MySQL **refuse** la suppression (à cause du `ON DELETE RESTRICT` posé sur la FK de `ORDONNANCE`) et lève une exception de code **1451**. Sans ce garde-fou, supprimer un patient effacerait silencieusement le lien de ses ordonnances — perte de données médicales.

Le filtrage `catch (SQLException ex) when (ex.Number == 1451)` permet de traiter spécifiquement la violation de clé étrangère, distincte des autres erreurs SQL.

À quoi ressemble un catch filtré ? Le `when` ajoute une condition : ce `catch` ne s'active **que** si la condition est vraie. Schéma illustratif :

```
try
{
    _patientController.SupprimerPatient(...);
}
catch (SQLException ex) when (ex.Number == 1451)
{
    // n'attrape QUE l'erreur de clé étrangère
    ...
}
catch (SQLException ex)
{
    // attrape toutes les autres erreurs SQL
    ...
}
```

Sans le `when`, tu serais obligé de tester `ex.Number` à l'intérieur du `catch` avec un `if`.

Flag C2 — Debug

Un camarade a écrit un `ModifierPatient` **sans clause WHERE**. Le code compile et s'exécute sans erreur. Explique en une phrase l'effet sur une base de 10 patients, et donne le nombre de lignes que `ExecuteNonQuery` retournera.

Chapitre 7 — CRUD Ordonnance (transactionnel)

L'objet métier central. Une ordonnance = un médecin + un patient + plusieurs lignes de prescription. Son enregistrement touche **deux tables** (ORDONNANCE et CONTENIR) en N+1 requêtes : une pour l'en-tête, une par médicament. Ces requêtes doivent être **atomiques** — toutes validées ensemble, ou toutes annulées — sinon on risque une ordonnance enregistrée sans ses médicaments, ou l'inverse. D'où la **transaction**, le concept neuf de ce chapitre.

7.1 — Contrôleurs de support

MÉTHODE	SIGNATURE	CONSIGNE
<code>MedecinController.ObtenirTousLesMedecins</code>	<code>List<Medecin></code> <code>ObtenirTousLesMedecins()</code>	SELECT, mapping en <code>Medecin</code>
<code>MedicamentController.ObtenirTousLesMedicaments</code>	<code>List<Medicament></code> <code>ObtenirTousLesMedicaments()</code>	SELECT, mapping en <code>Medicament</code>

7.2 — Le formulaire d'ordonnance

Consigne : crée `Views/OrdonnanceEditForm.cs`.

Contrôles : `cmbMedecin`, `cmbPatient` (combos), `dgvLignes` (grille des lignes éditable), labels, `btnEnregistrer` (OK), `btnAnnuler` (Cancel).

Champs privés : `_medecinController`, `_patientController`, `_medicamentController`, `_ordonnanceController`, et `_lignes` (une `BindingList<LignePrescription>`).

MÉTHODE PRIVÉE	SIGNATURE	CONSIGNE
Chargement	<code>OrdonnanceEditForm_Load(object, EventArgs)</code>	Remplit les combos (<code>DisplayMember = "Presentation"</code> , <code>ValueMember = "Id"</code>), lie <code>dgvLignes</code> à <code>_lignes</code> , configure les colonnes
Colonnes	<code>ConfigurerColonnesLignes()</code>	Une colonne <code>ComboBox</code> pour le médicament, deux colonnes texte pour posologie et durée

`BindingList<T>` notifie automatiquement la grille à chaque ajout/retrait de ligne — contrairement à `List<T>`. `DisplayMember` désigne la propriété affichée dans une combo (ici la méthode `Presentation()` du chapitre 1), `ValueMember` la propriété servant de valeur (`Id`).

Concrètement pour la grille des lignes : on lie `dgvLignes.DataSource = _lignes`. Quand l'utilisateur saisit une ligne (médicament + posologie + durée), elle s'ajoute à `_lignes`, et la grille se met à jour seule. À l'enregistrement, on parcourt `_lignes` pour construire l'ordonnance. La colonne « médicament » de la grille est une `DataGridViewComboBoxColumn` alimentée par `ObtenirTousLesMedicaments()` — l'utilisateur choisit dans une liste déroulante, pas en tapant un code à la main.

HANDLER	SIGNATURE	CONSIGNE
Enregistrer	<code>btnEnregistrer_Click(object, EventArgs)</code>	Valide (médecin + patient sélectionnés, au moins une ligne). Construit l' <code>Ordonnance</code> , ajoute les lignes, appelle <code>AjouterOrdonnance</code> .

7.3 — Le contrôleur transactionnel

MÉTHODE	SIGNATURE	CONSIGNE
<code>OrdonnanceController.AjouterOrdonnance</code>	<code>int AjouterOrdonnance(Ordonnance ord)</code>	Transaction : INSERT dans ORDONNANCE (récupère le numéro via <code>LAST_INSERT_ID</code>), puis un INSERT dans CONTENIR par ligne. <code>Commit</code> si tout passe, <code>Rollback</code> + <code>throw</code> sinon. Retourne le numéro d'ordonnance.

Pattern transactionnel (à mémoriser) :

À quoi ressemble une transaction ? On ouvre la transaction, on enchaîne les écritures, puis soit `Commit` (tout valider) soit `Rollback` (tout annuler) en cas de problème. Schéma illustratif :

```
using (var connexion = DbConnexion.Ouvrir())
using (var transaction = connexion.BeginTransaction())
{
    try
    {
        // INSERT ordonnance, puis INSERT de chaque ligne
        ...
        transaction.Commit(); // tout passe → on valide
    }
    catch
    {
        transaction.Rollback(); // un échec → on annule tout
        throw;                  // on propage l'erreur
    }
}
```

Chaque commande de la transaction se construit en lui passant la transaction : `new MySqlCommand(sql, connexion, transaction)`.

Les deux requêtes de la transaction : d'abord l'ordonnance (qui donne son numéro), puis chaque ligne.

```
-- 1) l'en-tête de l'ordonnance
INSERT INTO ORDONNANCE (dateEmission, numMedecin, numPatient)
VALUES (@date, @numMedecin, @numPatient);
SELECT LAST_INSERT_ID();           -- → on récupère numOrdonnance

-- 2) répété pour chaque médicament prescrit
INSERT INTO CONTENIR (numOrdonnance, codeMedicament, posologie, dureeJours)
VALUES (@numOrdonnance, @codeMedicament, @posologie, @dureeJours);
```

Le `numOrdonnance` obtenu en (1) est réinjecté comme paramètre dans chaque (2). Si l'une des requêtes (2) échoue, le `Rollback` annule **aussi** le (1) déjà exécuté.

Atomique signifie : les N+1 INSERT (l'ordonnance + ses lignes) forment **une seule opération indivisible**. Si l'un échoue, `Rollback` annule **tout**, y compris les INSERT déjà exécutés. On ne laisse jamais une ordonnance à moitié enregistrée. Le `throw;` après le `Rollback` propage l'erreur à l'interface pour qu'elle prévienne l'utilisateur — sans lui, l'app dirait « enregistré » alors que rien ne l'est.

Côté `PatientListForm` : ajoute `btnNouvelleOrdonnance` qui ouvre `OrdonnanceEditForm` en modale.

Flag E1 — Preuve du rollback

Modifie temporairement le contrôleur pour qu'il lève une exception sur la **2^e ligne** insérée. Crée une ordonnance à 2 médicaments. Vérifie dans phpMyAdmin qu'**aucune** ligne n'a été ajoutée ni dans ORDONNANCE ni dans CONTENIR, malgré le premier INSERT réussi. Retire la modif. Rends ta confirmation + une phrase d'explication.

Chapitre 8 — Finalisation du projet

Les fonctionnalités qui bouclent l'application : authentification du médecin, consultation des ordonnances, modification, export PDF. Aucun mécanisme radicalement neuf — c'est de la réutilisation des patterns des chapitres 4 à 7, appliqués à de nouveaux écrans.

8.1 — Authentification médecin

ÉLÉMENT	SIGNATURE / CONSIGNE
LoginForm (Views)	Form avec <code>txtRPPS</code> , <code>txtMotDePasse</code> (PasswordChar activé), <code>btnConnexion</code>
AuthController.Authentifier	<code>Medecin Authentifier(string numeroRPPS, string motDePasseClair)</code> — récupère le médecin par RPPS, vérifie le hash bcrypt du mot de passe. Retourne le médecin si OK, <code>null</code> sinon.
Hachage	Installe <code>BCrypt.Net-Next</code> . Le mot de passe en base est un hash <code>\$2a\$. . .</code> , jamais en clair.

Sécurité (bloc 3) : on ne compare jamais un mot de passe en clair. On hache la saisie et on compare au hash stocké, via `BCrypt.Verify`.

La requête d'authentification : on récupère le médecin par son RPPS (jamais par mot de passe dans le SQL).

```
SELECT numMedecin, nom, prenom, motDePasse
FROM MEDECIN WHERE numeroRPPS = @rpps
```

On récupère d'abord le médecin et son hash, **puis** on vérifie le mot de passe côté C# avec `BCrypt.Verify(saisie, hashStocké)`. On ne met **jamais** le mot de passe dans la clause `WHERE` : ce serait une comparaison en clair, et ça ouvrirait une injection sur le champ mot de passe.

8.2 — Liste et détail des ordonnances

ÉLÉMENT	SIGNATURE / CONSIGNE
<code>OrdonnanceController.ObtenirOrdonnancesParPatient</code>	<code>List<Ordonnance></code> <code>ObtenirOrdonnancesParPatient(int numPatient)</code> — sans les lignes (chargement léger)
<code>OrdonnanceController.ObtenirOrdonnance</code>	<code>Ordonnance ObtenirOrdonnance(int numOrdonnance)</code> — avec ses lignes (JOIN CONTENIR + MEDICAMENT)
<code>OrdonnanceListForm</code> (Views)	Deux grilles : <code>dgvOrdonnances</code> (haut) et <code>dgvLignes</code> (bas). L'événement <code>SelectionChanged</code> de la grille du haut rafraîchit celle du bas.
<code>OrdonnanceDetailForm</code> (Views)	Affiche une ordonnance complète, ouverte par double-clic depuis la liste

Le motif maître-détail : deux grilles synchronisées. Quand on sélectionne une ordonnance dans la grille du haut, on lit son `Id` via `DataBoundItem`, on charge ses lignes, et on les affiche dans la grille du bas. C'est le même réflexe `DataBoundItem` que pour le double-clic patient, appliqué à l'événement `SelectionChanged`. On distingue ici **chargement léger** (la liste, sans les lignes) et **chargement complet** (le détail, avec les lignes via `JOIN`) — on ne charge les lignes que quand on en a besoin, pour ne pas surcharger la liste.

8.3 — Modification d'ordonnance

MÉTHODE	SIGNATURE	CONSIGNE
<code>OrdonnanceController.ModifierOrdonnance</code>	<code>void</code> <code>ModifierOrdonnance(Ordonnance ord)</code>	En transaction : DELETE des lignes existantes dans <code>CONTENIR</code> , puis ré-INSERT des nouvelles. Stratégie « efface et réécrit », plus simple qu'un diff ligne à ligne.

Ajoute un constructeur `OrdonnanceEditForm(Ordonnance ord)` pour le mode modification, sur le modèle de `PatientEditForm`.

8.4 — Export PDF d'une ordonnance (optionnel)

ÉLÉMENT	CONSIGNE
<code>Services/OrdonnancePdfService</code> (nouveau dossier <code>Services/</code>)	Installe <code>QuestPDF</code> . Méthode <code>Generer(Ordonnance ord, string cheminFichier)</code> qui produit un PDF avec en-tête médecin, identité patient, tableau des lignes.

À quoi ressemble une génération QuestPDF ? On décrit le document par blocs imbriqués (page, en-tête, contenu), puis on génère le fichier. Schéma illustratif (très réduit) :

```
public void Generer(Ordonnance ord, string chemin)
{
    Document.Create(doc =>
    {
        doc.Page(page =>
        {
            page.Header().Text($"Dr {ord.Prescripteur.Nom}");
            page.Content().Column(col =>
            {
                col.Item().Text($"Patient : {ord.Patient.Nom}");
                // une ligne par médicament prescrit
                ...
            });
        });
    });
    .GeneratePdf(chemin);
}
```

Les `...` représentent la boucle sur `ord.Lignes`. La doc `QuestPDF` détaille les conteneurs (`Row`, `Table`, `Column`) à utiliser pour la mise en page.

Flag E2 — Authentification

Crée Dr DURAND avec un mot de passe haché. Vérifie qu'on **ne peut pas** se connecter avec le mot de passe en clair stocké tel quel, mais qu'on **peut** avec le bon mot de passe via `BCrypt.Verify`. Affiche le hash généré (il commence par `$2a$`). Rends-le.

Récapitulatif final du projet

Modèles (7) : `Personne`, `Patient`, `Medecin`, `Ordonnance`, `LignePrescription`, `Medicament`, `Allergie`

Contrôleurs (6) : `PatientController`, `AllergieController`, `MedecinController`, `MedicamentController`, `OrdonnanceController`, `AuthController`

Forms (6) : `PatientListForm`, `PatientEditForm`, `OrdonnanceEditForm`, `OrdonnanceListForm`, `OrdonnanceDetailForm`, `LoginForm`

Les trois patterns à maîtriser pour l'oral E6

1. **Lecture** : triple `using` + `ExecuteReader` + boucle de mapping
2. **Écriture simple** : `ExecuteNonQuery` (UPDATE/DELETE) ou `ExecuteScalar` + `LAST_INSERT_ID` (INSERT avec retour d'id)
3. **Écriture multi-tables** : transaction `BeginTransaction` / `Commit` / `Rollback` + `throw`

Les trois réflexes sécurité (bloc 3)

1. Toute donnée utilisateur passe par un **paramètre**
2. Les mots de passe sont **hachés** (bcrypt), jamais en clair
3. Les **contraintes FK** protègent l'intégrité (RESTRICT/CASCADE selon le sens métier)

Compétences mobilisées (référentiel BTS SIO SLAM)

Bloc 2 • Concevoir et développer une solution applicative → Analyser les besoins fonctionnels et techniques → Développer des applications en respectant les standards de programmation → Intégrer et tester les fonctionnalités avec des bases de données

Bloc 2 • Gérer les données → Concevoir et administrer des bases de données relationnelles → Écrire des requêtes pour manipuler ou extraire des données → Assurer l'intégrité, la cohérence et la sauvegarde des données

Bloc 2 • Assurer la maintenance corrective ou évolutive d'une solution applicative → Identifier et corriger les dysfonctionnements dans les applications existantes

Bloc 3 • Assurer la cybersécurité d'une solution applicative et de son développement → Prendre en compte la sécurité dans un projet de développement → Mettre en œuvre et vérifier la conformité à un référentiel de sécurité (OWASP) → Protéger les données à caractère personnel (données patients)